SYSTEM DESIGN RESEARCH REPORT

Designing Systems That Move Money

Design trade-offs from 24 fintechs, banks and platforms, for principal engineers and architects

Marisha Sahay · November 2026 (draft) · v0.3

CONTENTS

1. Key findings
2. Scope and method
3. The reference architecture
4. Architecture profiles
5. Topology and resilience
6. Languages and frameworks
7. The design matrix
8. Decision 1: How to record money
9. Decision 2: Where atomicity lives
10. Decision 3: Partitioning and scaling the store
11. Decision 4: The data access layer
12. Decision 5: Service and event boundaries
13. Decision 6: How to know the money is right
14. Migrating a live money system
15. Heuristics for architects
16. Limitations
17. References

Key findings

This report reviews the architecture and design of the systems that move money at 24 fintechs, banks and platforms, using only what they have published themselves. It is written for principal engineers and architects. Part I covers system architecture. Part II covers the design decisions inside it: the options companies took, what each bought, and what it cost.

Architecture

1. **Very different stacks converge on eight components, and only one of them owns money.** Edge, orchestration, rail gateways, ledger, balance store, event backbone, reconciliation and data platform appear across the sample. Balances, events and warehouse tables are all derived from the ledger. Uber names exactly two sources of truth: "the money order store and the entity balance store" [1, 2].
2. **The network edge stays physical.** Cloud-first banks still run their own data centers for card scheme and bank rail connections: Monzo, Starling, Wise and Nubank. Nubank authorizes cards without any cloud round trip [3, 4, 5, 6].
3. **The strongest isolation partitions whole stacks, not databases.** Ant Group routes users to units at the network entry, across three regions and five data centers, with a stated RPO of 0. Nubank clones its entire infrastructure per customer partition. Shopify forbids any action that crosses pods [7, 6, 8].
4. **Regional failure is designed for, and tested by reality.** In the October 2025 us-east-1 outage, Cash App kept running from us-west-2, while Square's European sellers saw up to 30% of transactions fail. Capital One removed database failover entirely, and Monzo keeps a separately written standby bank on a second cloud [9, 10, 11].

Design decisions

5. **Money is stored as entries, never as an overwritten balance.** The open questions are where the zero-sum invariant runs, how to keep balance reads fast, and how to handle hot accounts. Uber batches writes to hot accounts for a 10x gain, and Ant built a separate store for them [2, 12, 13, 14].
6. **Atomicity is kept local.** No company lets a money transaction span shards under two-phase commit. Cash App keeps each one on a single customer's shard, Starling inside one service's database, and Monzo gives up transactions and repairs afterwards [15, 16, 3].
7. **A narrow data access layer is what makes the database replaceable.** Klarna swapped Mnesia for PostgreSQL under its own access layer with no downtime. Stripe exposes only "a minimal battle-tested set of

functions". ORMs survive scale when sharding happens beneath them [17, 18, 15].

8. **Service counts vary more than 100x, from about 30 at Starling to more than 4,000 at Nubank.** The rule that holds everywhere is one database per service or module [19, 20, 21].
9. **Correctness is three layers, and the last one is a number.** Invariants stop bad writes, idempotency makes retries safe, and reconciliation measures the result. Stripe reports over 99.9999% explainability of money movement [22, 23, 2].

Scope and method

The sample is 24 companies that move money at scale: payment processors, neobanks, consumer fintechs, five banks, and four platforms that run their own payment systems. Zalando, an online retailer, is included as a reference case for one data access pattern. Companies were chosen for size and for how much they publish about their engineering, which biases the sample (see Limitations).

The report has two parts. **Part I** covers system architecture: a reference architecture built from what these companies publish, profiles of the best-documented systems, how they are deployed for scale and resilience, and the languages and frameworks they are built in. **Part II** covers the six design decisions inside that architecture, from how money is recorded to how it is verified, followed by how companies migrated live systems and a set of design heuristics. Vendor selection, build-versus-buy, front-end stacks, analytics and security tooling are out of scope.

How sources were graded

Every claim rests on a source that was opened and read, not on a search result. Each source carries one of three grades:

- **Strong:** the company's own engineering blog, conference talk, documentation, open-source code, or regulatory filing.
- **Medium:** the company's job listings, or material the company took part in, such as a vendor case study. These are labeled in the text wherever they carry a claim.
- **Weak:** third-party coverage. Used only to find stronger sources. No claim here rests on a Weak source alone.

In total, 412 sources were consulted. 303 of them are Strong, and 386 were read in full or in the relevant part. The sources page lists every one, with its grade, its date, and whether it was read. Any source older than five years carries its year in the text, because designs change.

The reference architecture

> **Behind very different stacks, the companies in this sample converge on the same eight components.** They differ in which component holds the truth, which ones are synchronous, and where the hard boundaries sit. This section maps the components. Part II covers the decisions inside them.

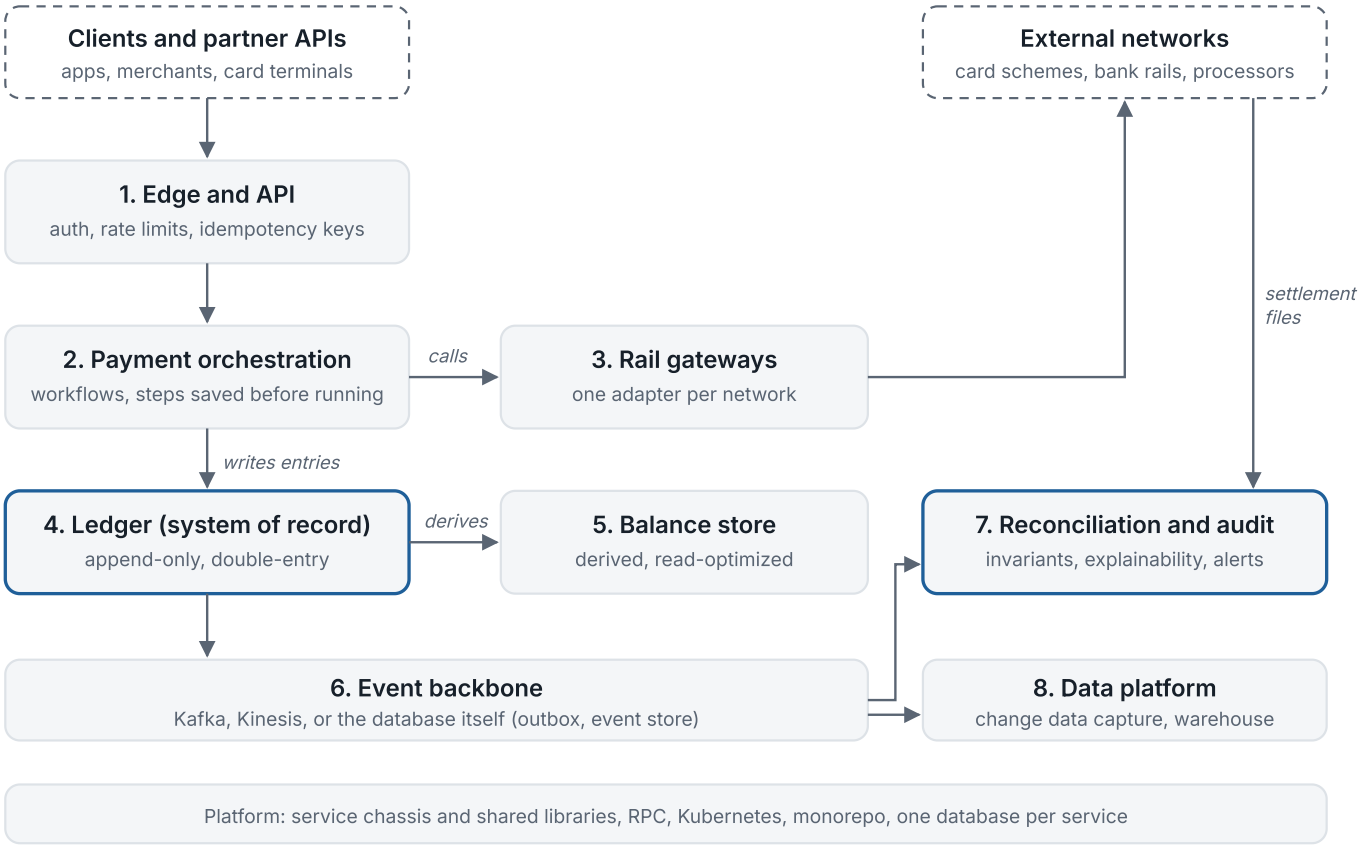


Figure 1. The components shared across the sample. Bold outlines mark the two components that hold or verify the truth about money. Dashed boxes are outside the system.

Component	Job	Where it is documented
1. Edge and API	Authenticate, rate-limit, and attach an idempotency key to every request that can move money	Adyen stores API idempotency keys in CockroachDB [24]; Airbnb's Billing Interface API fronts payments [25]
2. Payment orchestration	Run multi-step flows, save each step before running it, and resume after failure	Uber runs in-session payment flows on its Cadence workflow engine [1]; Starling stores each step before processing it [26]; Revolut has a dedicated "flow services" type [27]
3. Rail gateways	Translate internal payments into each external network's protocol	Monzo built its own Faster Payments gateway and runs it in its own data centres [28]; Airbnb has a Payments Gateway in front of processors [25]
4. Ledger	Record every movement as immutable, balanced entries	Stripe's Ledger [2], Uber's LedgerStore [29], Square's Books [30]
5. Balance store	Serve current balances fast, derived from the ledger	Uber's User Account Store holds authoritative balances [13]; Monzo precomputes balance blocks [12]; Nubank syncs authorizer balances through Kafka [6]
6. Event backbone	Carry state changes to everything downstream	Kafka at Nubank, PayPal and Wise [20, 31, 5]; Kinesis at Chime [32]; PostgreSQL at Revolut [27]
7. Reconciliation and audit	Prove that internal records match each other and the outside world	Uber's audit framework [33]; Airbnb's integrity pipeline [34]; Coinbase's Overseer [35]
8. Data platform	Copy online data out without loading the online databases	Shopify streams binlogs from 100+ MySQL shards with Debezium [36]; Stripe streams DocDB changes to Kafka and S3 [37]; Monzo runs reconciliation models in its warehouse [38]

Three things the model shows

The ledger is the only component that owns money. Balances, events and warehouse tables are all derived from it, and can be rebuilt. That is why the companies with the most mature systems put their strongest guarantees in the ledger and accept looser ones elsewhere. Robinhood, for example, lets balances lag the ledger until end of day [39].

The money path is short, and the rest is asynchronous. Only the edge, orchestration, ledger and gateway sit on the request path. Uber states the split directly: synchronous, in-session flows run on Cadence, and Kafka carries the rest asynchronously [1].

Verification is a component, not a test suite. Reconciliation has its own pipeline, data feeds and alerts at Uber, Airbnb, Stripe and Coinbase. It reads settlement files from outside the system, not only internal records [34, 2].

Architecture profiles

Thirteen companies publish enough to sketch their architecture end to end. Each profile gives the money path, the deployment topology, and the one choice that defines the design. Arrows mark the order a payment passes through components.

Payment platforms

Stripe. *Money path:* payments API → edge checks (rate limiters, idempotency keys and API versioning, described 2017 to 2020) → internal services through Stripe's own mesh proxy → DocDB → card networks. Ledger checks money movement after the fact instead of sitting on the request path [40, 41, 42, 43, 2]. *Topology:* AWS, with DocDB replica sets spread across availability zones and regions [18]. *Defining choice:* a verifying ledger off the hot path. "Ledger is our system of record and must remain an evergreen representation of truth" [2].

Adyen. *Money path:* stateless edge services → the payment processing layer, which calls risk, recurring and routing services before the payment goes out to the schemes → a local PostgreSQL queue → round-robin accounting clusters → streams to reporting [24, 22]. *Topology:* its own data centers in Europe, the US, India, Singapore, Japan and Australia, accepting payments at several locations at once, with no public cloud for payment processing [44]. *Defining choice:* a stateless, write-restricted edge. "All our externally exposed systems are prohibited from writing to central databases" (2021) [24].

Block (Cash App). *Money path:* Temporal workflows on every money transfer → each payment split into per-customer movements → Vitess routes each to its customer's MySQL shard → the Ledgering team's systems record it [45, 46, 47]. *Topology:* Cash App runs across availability zones in us-west-2, chosen "for its stability and low failure correlation". Square runs active in several AWS regions and shifts traffic between them regularly [9]. *Defining choice:* consistency over availability. "We will never compromise data consistency in exchange for availability" (2018) [15].

Uber. *Money path:* order insertion service → immutable order store and Kafka → order processing service → User Account Store, with hot accounts batched → entity changelog and LedgerStore → a payment integration interface → external providers [48, 13, 1]. *Topology:* active-active across regions as a stated goal (2018 and 2020), with standby cross-region consumers and balance storage on a multi-zone quorum. LedgerStore runs in Uber's own data centers [48, 49]. *Defining choice:* two named sources of truth. "The two shared sources of truth across all services are the money order store and the entity balance store" [1].

Airbnb. *Money path:* billing and pricing APIs → payment orchestration as a graph of idempotent steps over Kafka, using Orpheus → a processing gateway with a connector per processor. The ledger and reconciliation sit beside the path [50, 25]. *Topology:* AWS, with several processors integrated "to achieve global coverage and payments redundancy" (2018) [34]. *Defining choice:* "Every major workflow is divided into a directed acyclic graph (DAG) of retryable idempotent steps, each with well-defined behavior" [50].

Shopify. *Money path:* card fields in an iframe → CardSink, which encrypts and tokenizes the card → checkout in the Rails monolith and a background job → CardServer and a processor adapter → acquirer, network and issuer

[51]. *Topology:* shops grouped into pods with fully isolated datastores (2018). Each pod is active in one region and replicated to a second, and a whole pod can fail over [8, 51]. *Defining choice:* keeping card data away from the monolith. "We just don't let it see any card data in the first place" [51].

Neobanks and consumer fintech

Monzo. *Money path:* Mastercard → Monzo's own data centres → edge services on AWS → a Go card processor that calls account, financial crime and ledger services → the ledger → asynchronous events → daily reconciliation [3, 38]. *Topology:* AWS across availability zones, a few owned data centres for scheme connections, and Stand-in, a separately written backup bank of 18 services on Google Cloud [11]. *Defining choice:* a thin physical edge. "We run the absolute minimum compute possible to interface messages with our core platform" [3].

Nubank. *Money path (2017):* Mastercard → router → authorizer in Nubank's own data centers, with HSMs and fraud checks → Kafka to AWS → the customer's shard → double-entry service → balances back to the authorizers [6]. *Topology:* AWS, where each shard is a full copy of the infrastructure, plus a global section for cross-shard work. Authorization runs active-active in owned data centers [6]. *Defining choice:* authorization that does not depend on the cloud. "We don't need any cloud communication in order to authorize the transaction, that was important" [6].

Revolut. *Money path:* card network → card payments processor → card payment engine → Core Ledger and other core services → event store → financial crime and notification services [52]. *Topology:* Google Cloud and Kubernetes, with four availability tiers from 99.5% to 99.99%, and an availability tier required for every component [53]. *Defining choice:* fixed service types (frontend, flow and state). "Every service would have its own type predefined" [27].

Starling. *Money path (2018):* app → customer service stores the instruction and reduces the balance → payment service sends to Faster Payments → bank service records it in the ledger. Each step is keyed by an ID, stored first, and retried [26]. *Topology:* AWS from the start, with at least two instances of every service. Owned UK data centres hold the payment hardware and connect to AWS over Direct Connect. Engine now runs on Google Cloud and AWS [16, 4, 54]. *Defining choice:* the database as a work queue. "The database becomes the repository of what work must be done" [26].

Wise. *Money path:* quote → transfer → pay-in → ledgers → payouts run as tasks → direct connections to local schemes such as InstaPay, Zengin and PIX [5, 55, 56]. *Topology:* AWS with centralized networking and more than 20 Kubernetes clusters, plus physical data centres in the UK, Hungary and Australia for scheme connections [5]. *Defining choice:* side effects that run only after the business transaction commits. Tasks execute "only when business logic is successfully committed" [55].

Large scale and banks

Ant Group. *Money path:* request routed by user ID at the network entry to its unit → transaction service → payment service → accounting service, all in one TCC transaction → OceanBase [57, 58, 59]. *Topology:* units spread across three regions and five data centers, with a stated RPO of 0 and RTO under 30 seconds, and a Kubernetes cluster per unit [7, 57]. *Defining choice:* "The core idea of unitization is to shard user requests at the network entry point, not just at the data layer" [57].

Capital One. *Money path:* card processing at an external processor [60]. Transaction reads go from the mainframe through Lambda to a read-only DynamoDB store (2017) [61]. *Topology:* all AWS since 2020, chosen because "If one region were to go down, we can stand up in another region on AWS" [62]. The virtual card number service runs active-active across East and West regions on DynamoDB global tables (2021) [10]. *Defining choice:* no database failover. "The application writes to the database in its region and the data is replicated by AWS keeping the regions in sync" [10].

Topology and resilience

> **Cloud-first companies still keep the connection to the payment networks on hardware they own, and the most resilient designs isolate whole stacks, not just databases.** Recovery targets are almost never published. The best evidence of resilience in this sample comes from one real regional outage.

The network edge stays physical

Card schemes and bank rails connect through hardware security modules and scheme devices, so even cloud-native banks run a small physical tier:

- Monzo runs "a few data centers to integrate with some of the payment schemes", and keeps them minimal on purpose [3].
- Starling runs "multiple data centers across UK, and multiple MIP devices and multiple HSMs for redundancy", connected to AWS [4].
- Wise integrates with physical data centres in the UK, Hungary and Australia, and was an early user of AWS Outposts there [5].
- Nubank's card authorizer (2017) runs on physical servers alongside the Mastercard devices, in several live data centers, so that authorization needs no cloud round trip [6].

Adyen goes furthest: all its main systems run in its own data centers on several continents, with no public cloud for payment processing [44].

Cells: partition the whole stack

Design	Unit of isolation	What crosses units	Source
Ant Group "unitization"	A data center serving a range of user IDs, routed at the network entry	Calls that cross data centers are kept to a minimum	[57]
Nubank shards (2017)	A full copy of the infrastructure per customer partition	A global section of unsharded services, such as transfers between customers	[6]
Shopify pods (2018)	A set of shops on fully isolated datastores	Nothing: "we don't allow any actions to reach across pods"	[8]

The common idea is that the partition key chosen for the database (Decision 3) is also used to route traffic and deploy infrastructure. Ant states it directly: the approach "treats each data center as a massive, stateful database shard" [57]. The cost is a separate path for anything that spans units. Nubank needs a global section, and Shopify forbids cross-pod actions outright.

Active-active, active-passive, and an independent standby

Pattern	Who	Notes
Active-active across regions or sites	Ant Group, Adyen, Square, Capital One (virtual cards), Nubank (authorization)	Ant states RPO 0 and RTO under 30 seconds across three regions and five data centers
Active in one region, replicated to another	Shopify (per pod)	A whole pod fails over to its second region
Independent standby platform	Monzo (Stand-in)	Separately written services on a different cloud, holding minimal state. Not the system of record

Sources: [7, 44, 9, 10, 6, 51, 11].

The database is usually what makes active-active hard. In 2021 Capital One found application failover across regions "nearly instantaneous" except for PostgreSQL, where promoting a cross-region replica took ten to fifteen minutes. Moving that service to DynamoDB global tables removed database failover entirely [10]. Monzo's Stand-in

solves a different problem. Because its services are written separately and run on another cloud, it can cover failures that a replica of the same code on the same provider would share.

What a real outage showed

During the October 2025 AWS us-east-1 outage, Cash App kept running from us-west-2, a region Block had chosen "for its stability and low failure correlation". Square's European sellers, whose transactions were "primarily routed through us-east-1", saw up to 30% of transactions fail at peak [9]. Block's stated stance: "we design our systems with the assumption that outages like this will happen" [9]. The two outcomes, from one company in one incident, show the gap between multi-zone and multi-region designs more clearly than any target could.

Targets are rarely published

Ant Group is the only company in the sample that states RPO and RTO [7]. Stripe states a 99.9995% reliability target for its service mesh proxy and API availability above 99.999% [43, 2]. Revolut assigns every component one of four availability tiers [53]. For Uber, Airbnb, Wise, Adyen and Block, the research notes record no published recovery targets.

Languages and frameworks

> **The JVM is the most common runtime, but the language is rarely the decision that shapes the architecture.** More than half of the sample runs its core services on the JVM. Where a company explains its language, the language usually followed something else: the database, latency, or an existing codebase.

Runtime	Companies (main service language)
JVM	Adyen, Revolut, Starling, PayPal, Goldman Sachs, Ant Group, ING, Airbnb payments (Java); Wise, Cash App, N26 (Java and Kotlin); Nubank (Clojure)
Go	Monzo; Uber (60% of compute); Robinhood's newer services
Ruby	Stripe, Shopify, Chime; Coinbase's original application
Erlang	Klarna's core system, with Java for newer services

Sources: [24, 27, 19, 63, 64, 65, 34, 5, 66, 67, 68, 69, 70, 71, 72, 8, 73, 74, 17].

Adyen is "almost exclusively" Java, and Monzo states that "All of our services are written in Go" across about 2,800 services [24, 69]. Single-language estates like these make shared libraries and platform tooling cheaper, which matters more to the architecture than the language itself.

When the language followed something else

- **The database chose the language.** Nubank picked Datomic first: "So because of Datomic, we chose Clojure as the main programming language" [75].
- **Latency chose the language.** Robinhood began on Python and Django, then moved latency-sensitive trading and market data services to Go [76]. Its newest platform is Go on PostgreSQL [71].
- **The codebase kept the language.** Stripe kept Ruby and made it safer instead, building Sorbet, a type checker, for a Ruby codebase of more than 15 million lines [72].
- **The language stayed and the database moved.** Klarna's core system, KRED, has run on Erlang since 2004. Klarna migrated its 15 TB database from Mnesia to Aurora PostgreSQL over about three years, with no downtime, without rewriting the Erlang [17].

Frameworks are thin, and often in-house

Where companies describe their frameworks, they favor small, internal ones that encode platform decisions. Revolut runs "really vanilla Java services with our own internal DDD framework" [27]. Cash App builds on Misk, its own open-source Kotlin service framework [66]. Monzo's services share in-house Go libraries with Cassandra and

etcd built in [77]. Wise uses Spring Boot across more than 1,000 services, wrapped in an in-house chassis that sets defaults for security, observability, database access and Kafka [5]. The pattern is the same in each case: the framework is where the platform team puts the decisions it does not want every service team to make again.

The design matrix

One row per company, condensed from the research notes. "Not public" means no source was found, not that the design is absent.

Company	Transactional store	Data access	Ledger model	Atomic unit	Scaling approach	Sources
Stripe	DocDB, its own service on MongoDB	Minimal function set behind Go proxies	Immutable double-entry event log	Not public	2,000+ shards; online data movement	[18, 2]
Adyen	PostgreSQL, single databases of 100 to 300 TB	SQL-first: MyBatis, PL/pgSQL	Double-entry; templates proven to net to zero	Not public	Partition large tables; accounting sharded round-robin	[78, 79, 22]
PayPal	Oracle (2017); DynamoDB and Spanner in new services	Hera proxy; repository interfaces	Not public	One shard; no cross-shard 2PC; outbox	Custom sharding since 2009	[63, 80, 81]
Block (Cash App)	MySQL on Vitess	ORM: Hibernate	Immutable double-entry (Square Books, 2019)	One customer's shard	Vitess under the ORM	[15, 46, 30]
Monzo	Cassandra	Own Go data library	Double-entry; balance is the sum of entries	None; coherence service repairs	More nodes; time-bucketed balance blocks	[3, 12]
Revolut	PostgreSQL, also the event store	SQL-first: jOOQ	Not public	State and event in one database	Not public	[27, 82]
Wise	MariaDB, PostgreSQL	Shared in-house chassis	Several in-house ledgers	Idempotent tasks	Not public	[5, 55]
Starling	PostgreSQL, one per service	SQL-first: SQL on Java interfaces	Immutable double-entry	One service's database; no distributed transactions	About 30 services	[19, 16]
N26	PostgreSQL (from MySQL)	Not public	Not public	Not public	Not public	[67]
Nubank	Datomic, one per service	Immutable database	Immutable double-entry	One customer partition	Infrastructure cloned per customer partition	[6, 68]
Robinhood	PostgreSQL	Not public	Ledger eventually consistent with balances by day end	Relaxed on the ledger path	Application-level shards	[83, 39]
Coinbase	MongoDB; PostgreSQL	Connection-pooling proxy	Not public	Not public	Not public	[74]
Chime	Not public	Not public	Own ledger (ChimeCore)	Not public	Not public	[84]
Klarna	Aurora PostgreSQL (from Mnesia)	Own access layer (kdb)	Not public	One module, one database	Batched reads and writes	[17, 21]
Goldman Sachs	Relational, vendor not stated	Own ORM: Reladomo	Bitemporal history	Not public	Same schema across 100+ databases (2004)	[85, 86]
JPMorgan Chase	Not public	Not public	Not public	Not public	Not public	
Capital One	Mainframe core (2017)	Not public	Not public	Not public	Reads offloaded to a read-only DynamoDB copy	[61]
ING	Oracle; Cassandra	Not public	Not public	Not public	Group-wide Kafka event bus	[87, 88]
BBVA	DB2 on mainframe	Not public	Not public	Not public	Reads offloaded to Elasticsearch	[89]
Uber	LedgerStore on Docstore (MySQL-based)	Custom storage layer	Immutable double-entry money orders	One account, optimistic locking	Hot-account batching; hot and cold tiers	[1, 13, 49]
Airbnb	MySQL	ActiveRecord (legacy)	Immutable payment events	Idempotency keys on the primary	Idempotency tables sharded by key	[23, 50]
Shopify	MySQL, Vitess	ORM: ActiveRecord	Not public	One shop's pod	Shard by shop	[8, 90]

Company	Transactional store	Data access	Ledger model	Atomic unit	Scaling approach	Sources
Ant Group	OceanBase	SQL-first: MyBatis rules	Reserve, then confirm	TCC across services	Distributed SQL; separate hot-account store	[59, 58, 14]
Zalando (reference)	PostgreSQL	Stored procedures (2012 to 2015)	n/a	n/a	n/a	[91]

Three patterns are visible before any analysis. Every ledger that is described is built from entries. Where the atomic unit is described, it fits inside one database. And the banks fill most of the "Not public" cells, which is a finding in itself (see Limitations).

Decision 1: How to record money

> **Every company that describes its ledger stores money as entries and derives the balance from them. None stores a balance it overwrites.** Once that is settled, the real design work is in three places: where the zero-sum invariant is enforced, how a derived balance stays cheap to read, and what happens to accounts that are too hot to serialize.

Approach	Who	What it buys	What it costs
Immutable double-entry event log	Stripe	Every movement can be explained; Stripe reports over 99.9999% explainability	Corrections need revert-and-reprocess, not an UPDATE
Records that must net to zero before commit	Uber, Starling, Adyen	An unbalanced state cannot be written	More rows per payment (about 50 at Adyen)
Immutable database	Nubank	Audit history comes from the store itself	Poor fit for "fire hose writes", yet it became the default everywhere
Bitemporal rows	Goldman Sachs	Any past report can be reproduced after a correction	Two time axes in every query
Reserve, then confirm	Ant Group	Funds are held before a multi-service transaction commits	Every service implements try, confirm and cancel

Sources: Stripe [2]; Uber [1]; Starling [19]; Adyen [22]; Nubank [68, 6]; Goldman Sachs [86]; Ant Group [58].

Where the invariant runs

The zero-sum check can run at three points, and each moves cost somewhere different. Uber checks at write time: a money order whose debits and credits do not sum to zero is rejected before commit, and a correction is a new money order [1]. Adyen checks before deployment: journals can only be produced by templates, and every change to a template is verified automatically by proving that all combinations of inputs net to zero [22]. That costs nothing at runtime, but every new money flow needs a template. Stripe checks after the fact, by measuring how much money movement it can explain [2]. Mature systems combine these. Decision 6 covers the after-the-fact layer.

The read path is the cost nobody plans for

A derived balance gets slower as history grows. Monzo's ledger had no time component in its Cassandra partitions, so they "kept getting larger". By early 2023, P99 balance reads took 400 to 500 ms. Monzo reindexed entries into time-bucketed tables and precomputed sums of consecutive entries ("blocks"), which brought reads to about 200 ms [12]. A year earlier it had fixed a different read problem: each team computed balances with its own time axis, which "brought inconsistency between our data warehouse and our backend", so it replaced them with named balance definitions [92].

Robinhood hit the write side of the same problem. Its clearing ledger was a set of PostgreSQL tables updated by Kafka consumers in "a gigantic transaction with heavy locking". The redesign relaxed atomicity: ledger entries "only need to be eventually consistent with account balances at the end of the day" [39].

Hot accounts break per-account serialization

Serializing writes per account is the natural way to keep a balance correct, and it fails on hot accounts. At Uber some accounts needed more than 30 operations per second while the system handled 3 to 4. A batching service with 250 ms windows applies each batch as "an atomic update with optimistic locking", and Uber reports 10x throughput for hot entities [13, 1]. Ant Group went further and built Maxwell, a separate "hot-spot accounting database" for large sellers during shopping festivals. Ant says the usual workarounds (splitting accounts, buffering, non-real-time accounts) all have side effects, and reports "three orders of magnitude performance improvement" [14].

Corrections and deletion

Immutability changes how mistakes are fixed. Stripe: "we can't run simple queries to mutate the state; instead, we have to revert and reprocess prior operations" [2]. One cost no company here addresses publicly is erasing personal data from an immutable store under GDPR or Brazil's LGPD. Datomic supports a deletion feature called excision, but this research found no source on how any company in the sample uses it.

Decision 2: Where atomicity lives

> **No company in the sample lets a money transaction span shards under two-phase commit.** They shrink the atomic unit until one database can hold it, or they give up atomicity and repair inconsistency afterwards. Ant Group is the one company that runs a distributed transaction protocol, and it built its own.

Approach	Who	What it buys	What it costs
Every money transaction on one shard	Cash App	Full ACID inside MySQL	Payments had to be remodeled into per-customer "movements" before sharding
One database per service, no distributed transactions	Starling	Simple local transactions; failures handled by retrying	Every operation must be idempotent
Reserve, then confirm (TCC)	Ant Group	An atomic outcome across transaction, payment and accounting services	A protocol that every service must implement
No transactions; detect and repair	Monzo	Horizontal scale on Cassandra	A coherence service, locks and daily reconciliation
Relax to end-of-day consistency	Robinhood	Heavy locking removed from the ledger write path	Ledger and balances can disagree during the day
No cross-shard two-phase commit	PayPal	Simple failure modes in the sharding proxy	Applications must keep each transaction on one shard
State change and event in one commit (outbox)	PayPal (2026)	The event and the state cannot diverge	An outbox relay to run

Shrink the unit until it fits

Cash App shows the cost of the first option. Before its first shard split in November 2017, it split every payment into two "movements", one per customer, so each could live on its own customer's shard. The customer became the unit that never spans shards [46, 15]. The reason, in its words: "We will never compromise data consistency in exchange for availability" [15]. PayPal's Hera proxy enforces the same limit from below, by rejecting cross-shard two-phase commit [63]. Nubank's version is the largest unit of all: each customer partition gets a complete clone of the infrastructure [6].

Starling applies the rule at service level. Each service owns its own database, there are no distributed transactions, and the rule is to "do everything at least once and at most once": retry until success, and make every operation idempotent [16].

Give up atomicity, then pay to repair

Monzo chose Cassandra and says it "traded off the ability to have transactional consistency" [77]. The repair machinery is the price. Reads use quorum, writes go to index tables before the canonical table, and a separate coherence service detects and fixes inconsistencies [3]. Robinhood made the narrower trade on one path only. It relaxed consistency between ledger entries and balances to end of day, while noting that "if accounting ledgers are stale, downstream operations like trade reconciliation or settlement will be blocked" [39].

When a distributed transaction is unavoidable

Ant Group's escrow payments touch transaction, payment and accounting services, and it coordinates them with TCC. In the Try step, the accounting service moves the user's available funds into "pre-frozen funds". Confirm consumes the frozen funds and credits the recipient [58]. That source is a 2019 engineer's blog post, machine-translated, and graded Medium. Ant built the coordinator itself and has sold it externally since 2013 [93].

Make state and events agree

When a state change must also publish an event, PayPal's 2026 onboarding service commits both in one transaction (an outbox), and stores idempotency keys with their responses in the same database [81]. Revolut gets the same guarantee by using PostgreSQL as both database and event store (Decision 5).

Decision 3: Partitioning and scaling the store

> **Partition by whoever owns the money, and exhaust vertical scale, partitioning and read offloading before you shard.** The companies that sharded early paid with remodeling. The ones that avoided distributed databases did so to protect their data access code.

Choosing the partition key

Partition key	Who	What it buys	What it costs
Customer	Cash App, Nubank	A customer's money stays in one shard	Payments between customers need two records, one per shard
Shop	Shopify	Each shop is isolated in a "pod"	The sharding key must flow through every query, which needed ORM patches
Round-robin	Adyen (accounting)	Any shard can take any write; easy to add shards	Queries for one merchant fan out across all shards

Shopify's pods and its later move to Vitess both shard by shop. The Vitess move required "patches to ActiveRecord" to pass the sharding key through queries, which the team "wanted to avoid" [8, 90]. Adyen chose the opposite of locality on purpose. When it sharded its accounting database, it routed writes round-robin, not by merchant: "we decided the benefits domain knowledge routing would offer were not worth the loss of flexibility and increased complexity" [22].

How to shard

- **In the application.** Robinhood sharded its brokerage system from 1 to 10 shards at the application level. It had considered Spanner, DynamoDB, CitusDB and CockroachDB, and rejected them because they "would probably require a more invasive rewrite of our core business logic, and data access layers" [83].
- **In a proxy.** Cash App and Shopify's Shop app use Vitess under their ORMs [15, 90]. PayPal built Hera, its own sharding and connection proxy, and has used custom sharding since 2009 [63, 80].
- **In a database service.** Stripe built DocDB, a database service on MongoDB, which serves more than 5 million queries a second across more than 2,000 shards [18].

- **In a distributed database.** Ant Group built OceanBase, which has carried all of Alipay's core database traffic since 2016 [59].

Before sharding

Vertical scale goes further than most teams assume. Stripe scaled single shards vertically to tens of terabytes before building online data movement to scale out [18]. Adyen runs single PostgreSQL databases of 100 to 300 TB, and partitions any table that might exceed 100 GB [79, 94].

Moving reads elsewhere buys the most time:

- Adyen split a reporting database off its accounting database before sharding accounting [22].
- Capital One fed a read-only DynamoDB store from its mainframe core (2017), because "close to 80% of the traffic was related to reading transactions" [61].
- BBVA put Elasticsearch in front of DB2, so that "as soon as the customer logged in, the system was already querying Elastic, not DB2" [89]. That source is a vendor case study.
- Uber kept only 12 weeks of hot data in DynamoDB and moved older ledger data to blob storage [49].

Sharding is not the end state either. Nubank shards by customer, and says the scheme "worked well for a time but eventually hit physical limits" [95].

Decision 4: The data access layer

> **The ORM versus SQL debate has four answers in this sample, not two. The layer's most valuable property is narrowness.** The companies that changed databases or shards without rewriting their services all had one thing in common: data access went through a single, deliberately narrow layer.

Pattern	Who	What it buys	What it costs
ORM	Cash App (Hibernate), Shopify (ActiveRecord)	Productivity on schemas the service owns	Sharding keys must be patched through it; generated SQL is harder to bound
SQL-first	Adyen (MyBatis, PL/pgSQL), Revolut (jOOQ), Starling (SQL on Java interfaces), Alibaba (MyBatis rules)	Exact SQL on very large tables; schema-checked queries	More code per CRUD operation
Custom data layer or proxy	Stripe, Monzo, Uber, PayPal, Klarna	A narrow, controlled set of operations; the store can change underneath	A platform team to build and run it
Immutable database	Nubank (Datomic)	History and audit are built in	One store's limits apply everywhere

Sources: [46, 90, 78, 82, 19, 65, 18, 77, 63, 17, 6].

Narrow layers make the database replaceable

Klarna moved its 15 TB core from Mnesia to PostgreSQL over about three years, with no downtime, because business logic had not called Mnesia directly for years. All access went through kdb, an in-house layer that kept its API and gained a PostgreSQL backend [17]. PayPal's 2026 onboarding service goes further: business code sees only repository interfaces, and the backing store, DynamoDB or Spanner, is chosen at build time [81].

Stripe makes narrowness a reliability feature. DocDB exposes a minimal set of database functions behind Go proxies: "By exposing only a minimal battle-tested set of functions to our engineers, we ensure that these unintended performance and reliability issues don't materialize" [18]. Proxies also absorb load the application cannot control. Coinbase wrote a connection pooler because its "high scale Rails app" caused MongoDB connection storms [74], and Shopify routes MySQL traffic through ProxySQL [90].

ORMs survive scale when the scaling happens underneath

Cash App kept Hibernate and sharded MySQL with Vitess under it [46, 15]. Shopify kept ActiveRecord through pods and Vitess, at the cost of patches it hoped Rails composite keys would remove [90, 96]. Mixing styles is normal: 2026 Block job listings name Hibernate for Cash App Banking, jOOQ for the Ledgering team, and iBatis for Cash App Taxes [97, 47, 98].

The sources disagree about stored procedures

Zalando routed all Java database access through versioned PostgreSQL stored procedures from 2012, "to achieve both high performance and implement a secure API layer" [91]. By 2015 its own slides listed the cost, "CRUD operations need too much code", and teams later chose their own persistence [99, 100]. Alibaba's coding manual bans stored procedures as "difficult to debug, extend and not portable" [65]. The OceanBase team concluded the opposite: "the stored procedure still has great value for OLTP applications" [101]. That is a database team writing about its own product. Adyen uses PL/pgSQL in production today [78].

Decision 5: Service and event boundaries

> **Service counts differ by more than 100x between companies doing similar work, and two companies moved back toward modular monoliths. The rule that holds everywhere is one database per service or module.**

Company	Granularity	Data ownership	Source
Nubank	More than 4,000 microservices	Each microservice has its own database	[20, 102]
Monzo	About 2,800 Go services	One Cassandra keyspace per service	[69, 3]
Wise	More than 1,000 services	In-house chassis sets database access defaults	[5]
Starling	"About 30-odd services"	Each service has its own database instance	[19, 16]
Klarna (payments)	Microservices merged into a modular monolith	"One service, one database"	[21]
Shopify	Modular monolith (2019)	Sharded by shop	[103]

Why the counts diverge

The reasons given point in opposite directions. Klarna merged its per-payment-method microservices because shared steps were duplicated across them, creating "a distributed monolith". The result has "one service, one database, and a structured, clean codebase" and handles over a million orders a day [21]. Shopify chose a modular monolith in 2019 to avoid the network, deployment and data-access costs of splitting its Rails application [103]. Starling runs a core bank on about 30 services in a monorepo, released several times a day [19]. At the other end, Monzo and Nubank put the platform work into shared libraries and tooling, so that a new service is cheap. Monzo, for example, builds Cassandra and etcd integration into its shared library [77].

The count is a consequence of how cheap a new service is to create and operate, not a target. The invariant across every model is that a service or module owns its data and nothing else writes to it.

Kafka, or the database you already run

Most of the sample moves events through Kafka. Nubank reports 72 billion events a day [20]. PayPal runs more than 1,500 brokers and has peaked at 1.32 trillion messages a day [31]. Wise uses Kafka for "most of our real-time data movement" [5].

Two neobanks chose PostgreSQL instead. Revolut uses it "as the database, as the event store and event stream". It explains why it skipped Kafka: "Because we know exactly how to deal with databases … We know exactly how to monitor them, how to scale them" [27]. Starling chose PostgreSQL notifications over Kafka [19]. The trade: one fewer

system to operate, and state and events that commit together, in exchange for the fan-out and throughput a dedicated log provides.

Whichever transport is used, consumers must assume duplicates. Nubank requires every Kafka message to be idempotent [6]. Wise's task engine detects the same task being created twice, typically from a redelivered Kafka message [55].

Decision 6: How to know the money is right

> **No company trusts a single mechanism.** Every company that describes its approach combines three layers: invariants that stop bad writes, idempotency that makes retries safe, and reconciliation that measures the result.

Layer	Mechanism	Who
Stop bad writes	Zero-sum check at commit; templates proven to net to zero; single-shard transactions	Uber, Adyen, Cash App
Make retries safe	Idempotency keys on the primary; deduplication tables; idempotent consumers	Airbnb, Wise, Starling, Shopify, Robinhood, Nubank, PayPal
Measure the result	Explainability rate; daily audits; continuous reconciliation from database logs	Stripe, Uber, Airbnb, Monzo, Coinbase

Idempotency details that matter

- **Keep keys on the primary database.** Airbnb's Orpheus library keeps idempotency tables "always read and written from a sharded master database (for consistency)", sharded by idempotency key [23]. For payments, Airbnb also disables replica reads, "trading off load on the main database for data integrity" (2015) [104]. Replica lag is how a retry turns into a double charge.
- **Store the response with the key.** PayPal stores idempotency keys with their responses in the same database, and expires them with a TTL or a deletion policy [81].
- **Record progress, not just completion.** Shopify saves each payment request as a database row recording which step has completed, so a retry resumes instead of starting over [105].
- **Make it a library.** Wise's open-source task engine guarantees that no task "is processed twice. We don't want to have missing payouts or double payouts", and its idempotence4j library records every action in the database [55, 106].

Reconciliation as a metric

- Stripe measures the share of money movement it can explain, and reports over 99.9999% [2].
- Uber's audit framework checks more than 1 billion accounting transactions a day [33].
- Airbnb compares every transaction "from day zero" across its own systems and its processors' settlement files [34].
- Monzo reconciles its ledger daily against bank statements [38].
- Coinbase's Overseer reconciles by reading database write-ahead logs, which cut detection time from up to 24 hours to under a minute [35].

The working definition this suggests: a money system is correct when its errors are detected quickly and explained, not when its code is assumed to be bug-free.

Migrating a live money system

> **The migrations in this sample were slow, layered and verified.** None was a cutover. Each one changed one thing at a time behind an abstraction, and compared the old and new paths until the numbers matched.

Technique	Who	What it looked like
Put an abstraction in first	Klarna	Mnesia swapped for PostgreSQL behind its kdb layer, over about three years, with no downtime
Remodel the data before splitting it	Cash App	Payments split into per-customer movements before the first shard split
Dual-write, compare, then cut over	Stripe (2017)	Four steps: dual-write, move reads, move writes, remove the old path; reads compared with Scientist
Shadow, backfill, validate	Uber	Shadow validation at 99.99% or better; idempotent backfills; checksums matched across regions
Move data online in small units	Stripe	Online data movement across DocDB shards with versioned gating

Sources: Klarna [17]; Cash App [46, 15]; Stripe [107, 18]; Uber [49, 108].

Prove equivalence, do not assume it

Klarna used property-based tests to check that the Mnesia and PostgreSQL backends of kdb behaved identically. It also added multi-read and multi-write operations, because one batched PostgreSQL select was much faster than many separate calls [17]. The abstraction had to grow a little to fit the new store.

Uber's ledger migrations show the scale such verification reaches. The 2021 phase backfilled more than 250 billion records (about 300 TB) at up to 1 million messages a second. It closed each sealing window only when checksums from two regions matched, and used Kafka offsets and sequence numbers for exactly-once processing [108]. The later move off DynamoDB targeted at least 99.99% completeness and correctness in shadow validation, checked offline against a DynamoDB dump, and required every backfill job to be idempotent [49].

Remodel first

Cash App could not shard payments as they were, because a payment touched two customers. Splitting each payment into two movements, one for the sender and one for the recipient, came first. Sharding came after [46]. The order matters: the data model decides what can be partitioned, so it has to change before the storage does.

Heuristics for architects

Each rule below is practiced by several companies in the sample. The parenthesis names where the evidence is.

Architecture

1. **Make the ledger the only component that owns money.** Treat balances, events and warehouse tables as derived and rebuildable, and put the strongest guarantees in one place (Uber, Stripe).
2. **Keep the synchronous path short.** Edge, orchestration, ledger and gateway are on the request path. Everything else consumes events (Uber's split between Cadence and Kafka; Adyen's asynchronous processing behind a stateless edge).
3. **Keep the edge stateless and unable to write to core data.** An internet-facing component that cannot write to central databases limits what an attacker or a bug can reach (Adyen).
4. **Put network connectivity on a thin physical tier, and keep decisions in the core.** Scheme connections need owned hardware. Business logic does not (Monzo, Nubank, Starling, Wise).
5. **Use one partition key for data, traffic and deployment.** Route requests to cells by the same key that shards the database, and design an explicit global path for the few operations that cross cells (Ant Group, Nubank,

Shopify).

6. **Pick regions for low failure correlation, and drill the failover.** Multi-zone is not multi-region, as one 2025 outage showed (Block).
7. **Take database failover off the critical path where you can.** Writes that stay in their own region and replicate continuously removed a 10 to 15 minute promotion step (Capital One).
8. **For the most critical customer functions, consider a standby built from different code on a different provider.** A replica shares the primary's bugs. An independent minimal platform does not (Monzo).

Design

1. **Store entries, derive balances, and design the balance read path on day one.** Summing history gets slow, and fixing it later means reindexing a live ledger (Monzo, Decision 1).
2. **Put the zero-sum invariant where it is cheapest to enforce, and enforce it more than once.** At build time for known flows (Adyen), at commit for everything (Uber), and in reconciliation afterwards (Stripe).
3. **Size the atomic unit to fit one database.** Choose the partition key so that a money transaction never needs a second shard. If a business operation spans two owners, split it into two records (Cash App).
4. **Treat hot accounts as a separate design problem.** Per-account serialization caps throughput. Batching (Uber, 10x) or a dedicated store (Ant, three orders of magnitude) are the documented answers.
5. **Exhaust vertical scale, partitioning and read offloading before sharding.** Single PostgreSQL databases of 100 to 300 TB run in production (Adyen). Most read traffic can move to a copy (Capital One, BBVA).
6. **Keep data access behind one narrow layer.** It is what made database swaps (Klarna), store portability (PayPal) and safe scaling (Stripe) possible without rewriting services. An ORM is acceptable inside that layer, as long as sharding happens underneath it (Cash App, Shopify).
7. **Give every service or module exclusive ownership of its data, and choose the service count by operating cost.** Thirty services (Starling) and four thousand (Nubank) both work. A distributed monolith does not (Klarna).
8. **Prefer the system your team already operates for events, until throughput forces a log.** PostgreSQL as an event store (Revolut) and Kafka peaking above a trillion messages a day (PayPal) are both valid at their scale. Either way, make every consumer idempotent.
9. **Keep idempotency keys and money reads on the primary.** Store the response with the key, and record the steps completed so a retry can resume (Airbnb, PayPal, Shopify).
10. **Measure correctness as a number.** An explainability rate or a daily audit count turns "we think it is right" into something a team can track (Stripe, Uber).
11. **Migrate behind an abstraction, remodel before you split, and verify by comparison.** Klarna's kdb, Cash App's movements, and Stripe's and Uber's dual paths all followed this order.

Limitations

Disclosure is uneven. Fintechs that recruit engineers through their blogs publish a great deal. Banks publish very little about their transactional systems. For JPMorgan Chase, Capital One, ING and BBVA, this research found no public source on how applications reach the database or on the ledger model. Several bank and Medium-hosted pages also refused automated reading, so a few relevant sources remain unread. "No public source found" in this report means exactly that. It is not evidence that a company does not use a technology.

What gets published is biased. Companies write about systems that worked and migrations that finished. Abandoned designs and quiet reversals rarely get a blog post, so the record over-represents success.

Some of the strongest sources are old. Several key sources date from 2012 to 2021. Their years are given in the text, and the per-company research notes record later evidence where it exists. A design described in 2017 may have changed since.

Job listings describe intent. A listing shows what a team works with or wants to hire for. It is a weaker signal than an engineering post about a running system, so listings are graded Medium and labeled where they carry a claim.

Some authors have a stake. A database team writing about its own database, or a vendor writing about a customer, has reasons to present it well. Those sources are flagged where they matter. Where two first-party sources disagree on a date or figure, the research notes record both.

References

1. Zero Sum by Design: 10 Years of Uber's Payments Platform. Uber Engineering Blog, 2026-08. Strong
2. Ledger: Stripe's system for tracking and validating money movement. Stripe (stripe.dev, Ilya Ganelin), 2024-02. Strong
3. Banking on Thousands of Microservices, QCon London 2023 (Suhail Patel; InfoQ published 2024-01). InfoQ, 2023. Strong
4. How Starling Built Their Own Card Processor, QCon Plus transcript (Rob Donovan, Ioana Creanga). InfoQ, 2022-12. Strong
5. Wise Tech Stack (2025 update) (Milon James); read via the publication's RSS feed because the article page returned 403. Wise Engineering (Medium), 2025-02. Strong
6. Architecting a Modern Financial Institution (Edward Wible, Rafael Ferreira, QCon SF 2017). InfoQ, 2017-12 (older than 5 years). Strong
7. Building a financial-grade transaction system from distributed to cloud-native (SOFAStack docs). Alibaba Cloud / Ant Group, 2026-03 (last updated; original date not shown). Strong
8. A Pods Architecture To Allow Shopify To Scale (2018). Shopify Engineering (Xavier Denis), 2018-03. Strong
9. Building for Resilience: How Block's Engineering Teams Kept Systems Running During the AWS Outage (Randy Wigginton). Block Engineering Blog, 2025-10. Strong
10. Moving to DynamoDB to increase application resiliency (Kelly Jo Brown). Capital One Tech, 2021-02. Strong
11. Tolerating full cloud outages with Monzo Stand-in (Daniel Chatfield, Andrew Lawson). Monzo, 2025-02. Strong
12. Speeding up our balance read time: the planning phase (Basma Taha, Nick Ng). Monzo, 2023-04. Strong
13. Building High Throughput Payment Account Processing. Uber Engineering Blog, 2026-03. Strong
14. Maxwell: a hardware and software highly integrated compute-storage system (arXiv 2112.11030). Ant Group, 2021-12. Strong
15. Sharding Cash. Cash App Code Blog (Jon Tirsén), 2018-11. Strong
16. Building a Reliable Cloud Bank in Java, slides (Jason Maude, March 2018). QCon London archive, 2018-03. Strong
17. The fellowship of the forgotten (KRED migration from Mnesia to Postgres; Onno Vos; read via the blog's RSS feed). Klarna Engineering, 2025-02-26. Strong
18. Stripe's DocDB: How Zero-Downtime Data Movement Powers Trillion-Dollar Payment Processing (QCon San Francisco 2025 talk, transcript published 2026-04). InfoQ (speaker Jimmy Morzaria, Stripe), 2025-11. Strong
19. Rampant Pragmatism: Growth and Change at Starling Bank, QCon London 2020 transcript (Dan Osborne, Martin Dow). InfoQ, 2020. Strong
20. Managing cloud limits (Engineering Meetup #10). Building Nubank, 2025-04-09. Strong
21. How micro should your microservices be? (Raya Rizk; read via RSS feed). Klarna Engineering, 2025-03-24. Strong
22. Design to Duty: The Accounting and Reporting Systems at Adyen. Adyen (Willem Pino), 2021-08. Strong
23. Avoiding double payments in a distributed payments system. Airbnb Tech Blog (Jon Chew, Ninad Khisti), 2019-04. Strong
24. Design to Duty: How we make architecture decisions at Adyen. Adyen, 2021-07. Strong
25. Scaling Airbnb's Payment Platform. Airbnb Tech Blog (Angela Zhu, Karen Kim), 2016-09. Strong
26. Building a Reliable Cloud-Based Bank in Java, QCon San Francisco 2018 transcript (Jason Maude). InfoQ, 2018. Strong
27. Unveiling the Tech Underpinning FinTech's Revolution, QCon London 2024 (Wojtek Ptak, Andrzej Grzesik). InfoQ, 2025-02. Strong
28. How we moved our Faster Payments connection in-house (Wayne Tsai). Monzo, 2020-05. Strong
29. How LedgerStore Supports Trillions of Indexes at Uber. Uber Engineering Blog, 2024-04. Strong
30. Books, an immutable double-entry accounting database service. Square Developer Blog (The Corner), 2019-10. Strong
31. Scaling Kafka to Support PayPal's Data Growth. PayPal Developer Blog (Monish Koppa), 2023-09. Strong
32. Bridging the Data Gap (Part I of Building a Streaming Platform at Chime; read via the publication's RSS feed). Life at Chime (Medium), 2022-11-15. Strong
33. Automated Audit Framework For Internet Scale Financial Transactions. Uber Engineering Blog, 2023-06. Strong
34. Measuring transactional integrity in Airbnb's distributed payment ecosystem. Airbnb Tech Blog (Ninad Khisti, William Betz), 2018-01. Strong
35. Real-time reconciliation with Overseer (Cedric Cordenier; read in full via the publication's RSS feed). The Coinbase Blog (Medium), 2022-09-06. Strong
36. Capturing Every Change From Shopify's Sharded Monolith (2021). Shopify Engineering (John Martin, Adam Bellemare), 2021-03. Strong
37. How Stripe's document databases supported 99.999% uptime with zero-downtime data migrations. Stripe (stripe.dev, Jimmy Morzaria and Suraj Narkhede), 2024-06. Strong
38. Processing payments safely at scale (Joost van Oorschot). Monzo, 2022-02. Strong
39. Part I: Scaling Robinhood Clearing Accounting. Robinhood Newsroom, 2022-04-11. Strong
40. Stripe's payments APIs: The first 10 years (2020, older than 5 years). Stripe (stripe.dev, Michelle Bu), 2020-12. Strong
41. Scaling your API with rate limiters (2017, older than 5 years). Stripe (Paul Tarjan), 2017-03. Strong
42. Designing robust and predictable APIs with idempotency. Stripe (Brandur Leach), 2017-02. Strong
43. Building a data plane from scratch: Stripe's own high-performance distributed proxy. Stripe (stripe.dev, Jack Loughran and Cara Mecozzi), 2026-08. Strong
44. Payments platform infrastructure. Adyen, 2026-11 (accessed; undated page). Strong
45. Great tech is not enough: building trust to get the most out of Temporal at Cash App (Replay 2024 session page, abstract and bios only; Nick Esposito, Riley Pruitt). Temporal, 2024-10. Strong
46. Remodeling Cash App Payments. Cash App Code Blog (Jesse Wilson), 2018-11. Strong
47. Senior Software Engineer, Ledgering, job listing via Built In NYC (removed 2026-09-01). Block, 2026. Medium
48. Revolutionizing Money Movements at Scale with Strong Data Consistency (2020, older than 5 years). Uber Engineering Blog, 2020-10. Strong
49. Migrating a Trillion Entries of Uber's Ledger Data from DynamoDB to LedgerStore. Uber Engineering Blog, 2024-04. Strong
50. Rebuilding Payment Orchestration at Airbnb. Airbnb Tech Blog (Bryon Ross and others), 2022-02. Strong
51. Shopify's Architecture to Handle the World's Biggest Flash Sales (QCon Plus May 2022, transcript). InfoQ (speaker Bart de Water, Shopify Payments), 2022-10. Strong
52. Taming Complexity in Distributed Systems: What's behind Revolut success, XT25 slide deck (Donato Lucia, Partner and Head of Technology; 28 slides, copyright 2025 Revolut). Revolut (hosted by JUXT), 2025. Strong
53. Extreme DevOps Automation, QCon London 2025 transcript (Sergio Amorim, DevOps Platform Developer Experience lead). InfoQ, 2025-08. Strong
54. Staff Infrastructure Engineer (GCP) - Engine by Starling (job listing, undated, accessed 2026-11). Starling Bank via Workable, 2026. Medium
55. transferwise/tw-tasks-executor README and docs (created 2019-05, archived 2025-11). Wise (GitHub), 2025-11. Strong

56. From Onboarding to Production: How I Built Dynamic Vendor Switching at Wise; read via the publication's RSS feed. Wise Engineering (Medium), 2026-11. Strong
57. From unitization to elastic architecture: Handling explosive traffic spikes on the Internet (SOFAStack docs). Alibaba Cloud / Ant Group, 2026-03 (last updated; original date not shown). Strong
58. Analysis of Applicable Models and Scenarios for TCC (English machine translation; older than 5 years). Apache Seata blog, author zhangthen (Ant Financial team), 2019-03. Medium
59. Development of OceanBase Database (official docs, V4.3.5). OceanBase, 2025-01. Strong
60. Form 10-K for fiscal year 2023 (Technology and Intellectual Property, Technology/Systems). Capital One via SEC EDGAR, 2024-02. Strong
61. Fintech: How serverless transactions better serve customers. Capital One Tech, 2017-11. Strong
62. How Capital One migrated to the cloud: Lessons from an 8-year journey. Capital One Software blog, 2024-05. Strong
63. Scaling DB Access for Billions of Queries Per Day @PayPal (QCon New York 2019, InfoQ transcript). InfoQ (speakers Petrica Voicu and Kenneth Kang, PayPal), 2019-09. Strong
64. goldmansachs/reladomo repository and README. Goldman Sachs (GitHub), 2016-09. Strong
65. Alibaba Java Coding Guidelines (English translation of an earlier edition). Alibaba Group, 2022-02 (last commit). Strong
66. cashapp/misk (README, module list, misk-jooq, misk-vitess and misk-spirit READMEs). Cash App (GitHub), 2026-11. Strong
67. Engineering at N26: a Tour of our Tech Stack and Architecture (Rafael Roman); read via the publication's RSS feed because the article page returned 403. N26 (InsideN26 on Medium), 2022-10. Strong
68. Nu Holdings Ltd. Form F-1 registration statement (Business, Risk Factors, Letter from founders). Nu Holdings / SEC EDGAR, 2021-11-01 (older than 5 years). Strong
69. How we run migrations across 2,800 microservices (Will Sewell). Monzo, 2024-08. Strong
70. Adopting Arm at Scale: Transitioning to a Multi-Architecture Environment. Uber Engineering Blog, 2025-02. Strong
71. How we evolved Robinhood's derivatives platform for a new market in under three weeks (Edmond Wong). Robinhood Careers Blog, 2026-06-09. Strong
72. Sorbet: Stripe's type checker for Ruby. Stripe (stripe.dev, Jake Zimmerman), 2022-03. Strong
73. Chime joins the Rails Foundation as a Contributing Member (quotes Chime co-founder Ryan King). Rails Foundation, 2025-07-15. Medium
74. coinbase/mongobetween: MongoDB connection pooler (README). GitHub (Coinbase), undated (accessed 2026-11). Strong
75. Lucas Cavalcanti on Nubank's architecture, Clojure and Datomic (InfoQ Podcast). InfoQ, 2021-08-16 (older than 5 years). Strong
76. SED 1136: Robinhood Engineering with Jaren Glover (transcript). Software Engineering Daily, 2020 (older than 5 years). Strong
77. Modern Banking in 1500 Microservices, QCon London 2020 (Matt Heath, Suhail Patel). InfoQ, 2020. Strong
78. Injecting metadata as comments in SQL queries. Adyen (Cosmin Octavian Pene), 2023-02. Strong
79. Operational hazards of managing PostgreSQL DBs over 100TB (PGConf.EU 2025 abstract and slides). PostgreSQL Europe (speaker Teresa Lopes, Adyen), 2025-10. Strong
80. Oracle Database Native Sharding: a Customer Perspective (slides by John Kanagaraj, PayPal Core Data Platform). Oracle (vendor-hosted PDF), 2017-02. Medium
81. One Codebase, Two Clouds. PayPal Developer Blog (Mounika Gampa), 2026-07. Strong
82. revolut-engineering/jooq-plugin: Gradle plugin for generating jOOQ classes using dockerized databases (created 2019-05, last commit 2026-02). Revolut (GitHub), 2026-02. Strong
83. How we scaled Robinhood's brokerage system for greater reliability (Edmond Wong, Nathan Ziebart). Robinhood Newsroom, 2021-06-25 (older than 5 years). Strong
84. Chime Financial, Inc. Form 10-K for fiscal year 2025. Chime / SEC EDGAR, 2026-03-06. Strong
85. Introducing Reladomo - Enterprise Open Source Java ORM, Batteries Included! (Mohammad Rezaei). InfoQ, 2017-03. Strong
86. Introducing Reladomo - Enterprise Open Source Java ORM, Batteries Included! (Part 2). InfoQ, 2017-06. Strong
87. ING Belgium Success Story. Temenos, n.d. Medium
88. The Evolution of Kafka at ING Bank (Timor Timuri, Richard Bras). Confluent (Kafka Summit London 2018), 2018. Strong
89. BBVA transforms global banking operations with Elasticsearch. Elastic, n.d. Medium
90. Horizontally scaling the Rails backend of Shop app with Vitess. Shopify Engineering (Hammad Khalid et al.), 2024-01. Strong
91. Java Wrapper for PostgreSQL Stored Procedures (Jan Mussler, PGConf.EU 2012 session abstract). PostgreSQL Europe, 2012-10. Strong
92. How we calculate balances (Basma Taha). Monzo, 2022-02. Strong
93. Benefits of distributed transaction (SOFAStack DTX docs). Alibaba Cloud / Ant Group, 2026-01 (last updated). Strong
94. A Deep Dive into Table partitioning Part 2: Partitioning at Adyen. Adyen (Derk van Veen and Cosmin Octavian Pene), 2023-10. Strong
95. 10 years of engineering at Nubank: Lessons from scaling to 122+ million customers. Building Nubank, 2025-08-20. Strong
96. How to Introduce Composite Primary Keys in Rails (2020). Shopify Engineering (John Arthorne), 2020-10. Strong
97. Staff Software Engineer, Cash App Banking, job listing. Block, 2026-11. Medium
98. Senior Software Engineer, Cash App Taxes, job listing. Block, 2026-11. Medium
99. Enabling development teams to move fast: PostgreSQL at Zalando (Valentine Gogichashvili and Oleksii Kliukin, PGConf.RU slides). PGConf.RU, 2015. Strong
100. Data Integration in a World of Microservices. Zalando Engineering Blog, 2015-09. Strong
101. OceanBase: A 707 Million tpmC Distributed Relational Database System (PVLDB 15(12)). OceanBase (Ant Group), VLDB Endowment, 2022. Strong
102. Software-driven data privacy: an innovative project journey. Building Nubank, 2024-04-22. Strong
103. Deconstructing the Monolith: Designing Software that Maximizes Developer Productivity (2019). Shopify Engineering (Kirsten Westeinde), 2019-02. Strong
104. Large Scale Payments Systems and Ruby on Rails. Airbnb Tech Blog (Michel Weksler; first published on nerds.airbnb.com 2015-02-26, reposted on Medium 2016-03-04), 2015-02. Strong
105. Building Resilient GraphQL APIs Using Idempotency (2019). Shopify Engineering (Todd Jefferson), 2019-08. Strong
106. transferwise/idempotence4j README (created 2020-05, release 2.0.0 in 2026-05). Wise (GitHub), 2026-05. Strong
107. Online migrations at scale. Stripe (Jacqueline Xu), 2017-02. Strong
108. How Uber Migrated Financial Data from DynamoDB to Docstore (2021, older than 5 years). Uber Engineering Blog, 2021-11. Strong

Every source consulted, including those not cited above, is listed on the sources page.